

Why robust verification?

- Many autonomous systems are **concurrent**, **stochastic**, and operate under **uncertain** dynamics.
- Transition probabilities are **learned** from limited or noisy data, or arise from model **abstraction**.
- Concurrent stochastic games (CSGs)** model multi-agent stochastic systems with simultaneous action choices.
- Standard CSG verification assumes exact transitions → unrealistic**

How do we model **uncertainty** in CSGs, and perform **robust verification (and synthesis)** on these models?

Model: Interval Concurrent Stochastic Games

- CSG:** $\mathcal{G} = (N, S, \bar{s}, A, \Delta, P)$ 0 zero-sum 1 nonzero-sum
- N players simultaneously choose actions
Joint action determines probabilistic transition
- (Minimax) value, optimal strategies**
Nash Equilibria (NE)
Σ Social-welfare optimal NE (SWNE):
NE that maximises players' total utility
- Robust CSG (RCSG):** $\mathcal{G} = (N, S, \bar{s}, A, \Delta, \mathcal{P})$ *uncertain transition function*
 $\mathcal{P}$ describes a set of possible transition functions
→ holds under all possibilities / worst-case
- Robust value, robust optimal strategies**
Robust NE (RNE)
Σ Robust SWNE (RSWNE):
NE that maximises worst-case total utility
- Interval CSG (ICSG):** $\mathcal{G} = (N, S, \bar{s}, A, \Delta, \check{P}, \hat{P})$
RCSG with interval uncertainty
- $$\mathcal{P}_{sa} = \left\{ P_{sa} \in \mathcal{D}(S) \mid \forall s'. P_{sa}(s') \in [\check{P}_{sas'}, \hat{P}_{sas'}] \right\}$$
- Objectives:** (in)finite-horizon probabilistic & reward reachability

Robust Bellman Equations

- Zero-sum:** nature adversarial against one player, e.g., minimises player 1's utility
- uncertainty* *nature's inner problem*
- $$V(s) = \sup_{\sigma_1 \in \mathcal{D}(A_1(s))} \inf_{\sigma_2 \in \mathcal{D}(A_2(s))} \left\{ r(s, \sigma) + \sum_{a \in A(s)} \sigma_{sa} \inf_{P_{sa} \in \mathcal{P}_{sa}} \sum_{s' \in S} P_{sas'} V(s') \right\}$$
- player 1's value* *mixed strategies for optimality*
- Nonzero-sum:** nature minimises total utility
- $$V_+(s) = \sup_{\sigma \in \Sigma_{\mathcal{G}}\text{-RNE}} \left[r_+(s, \sigma) + \sum_{a \in A} \sigma_{sa} \inf_{P_{sa} \in \mathcal{P}_{sa}} \left\{ \sum_{s' \in S} P_{sas'} V_+(s') \right\} \right]$$
- players' total value* *players optimise their own objectives*

Contributions

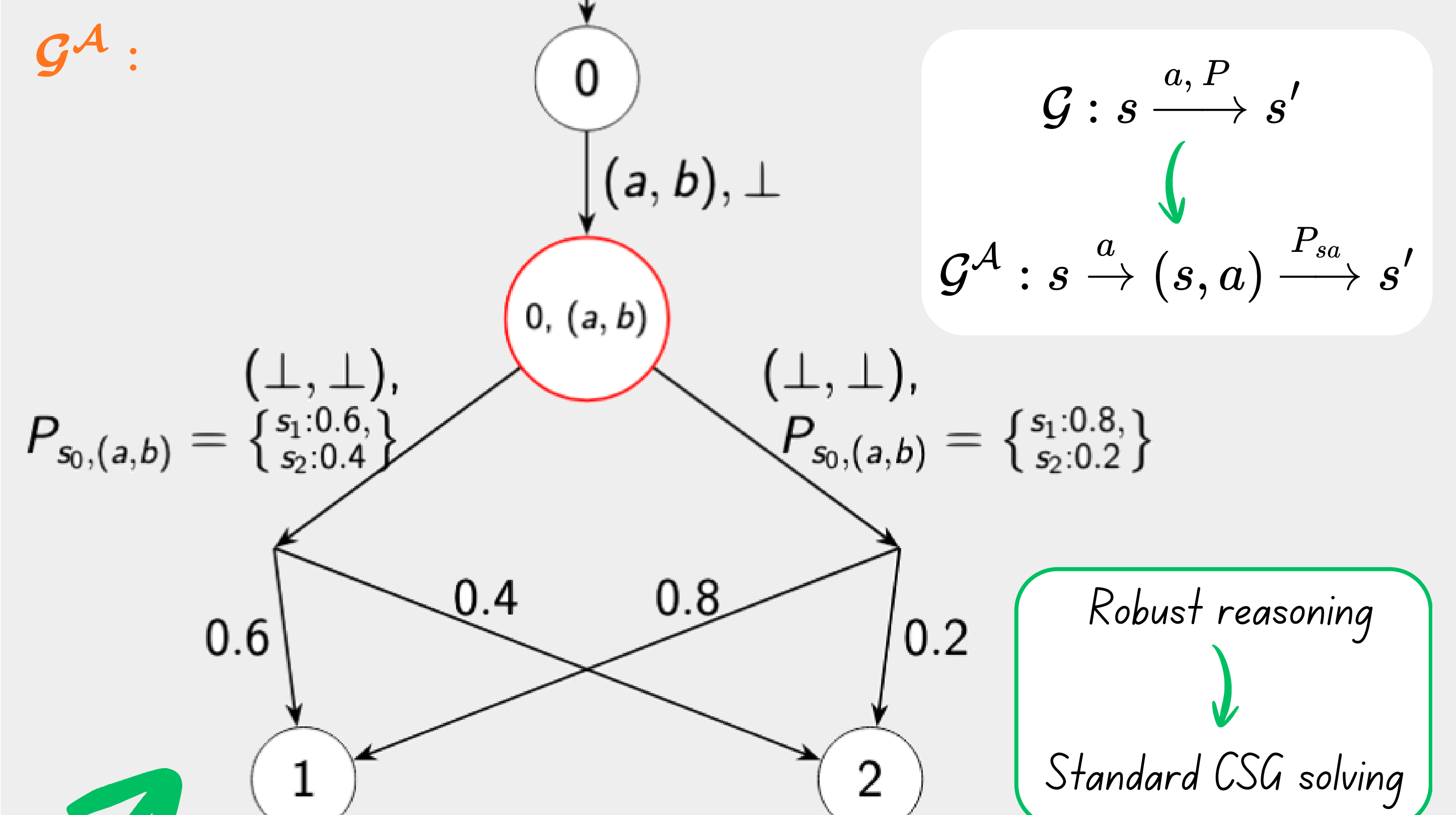
Introduced **RCSGs** (with focus on ICSGs) + **robust** solution concepts

- Value-preserving** reduction to **non-robust CSG** solving
- Player-/nature-first** value-equivalence
- Scalable** ICSG solver implemented in PRISM-games

Adversarial Expansion

Model **nature as player 3** who chooses transitions adversarially

- 0** Nature aligns with opponent → **2-player CSG** via "deviation MDPs"
- 1** Nature minimises social welfare → **3-player CSG + NE filtering**



Main Theoretical Results

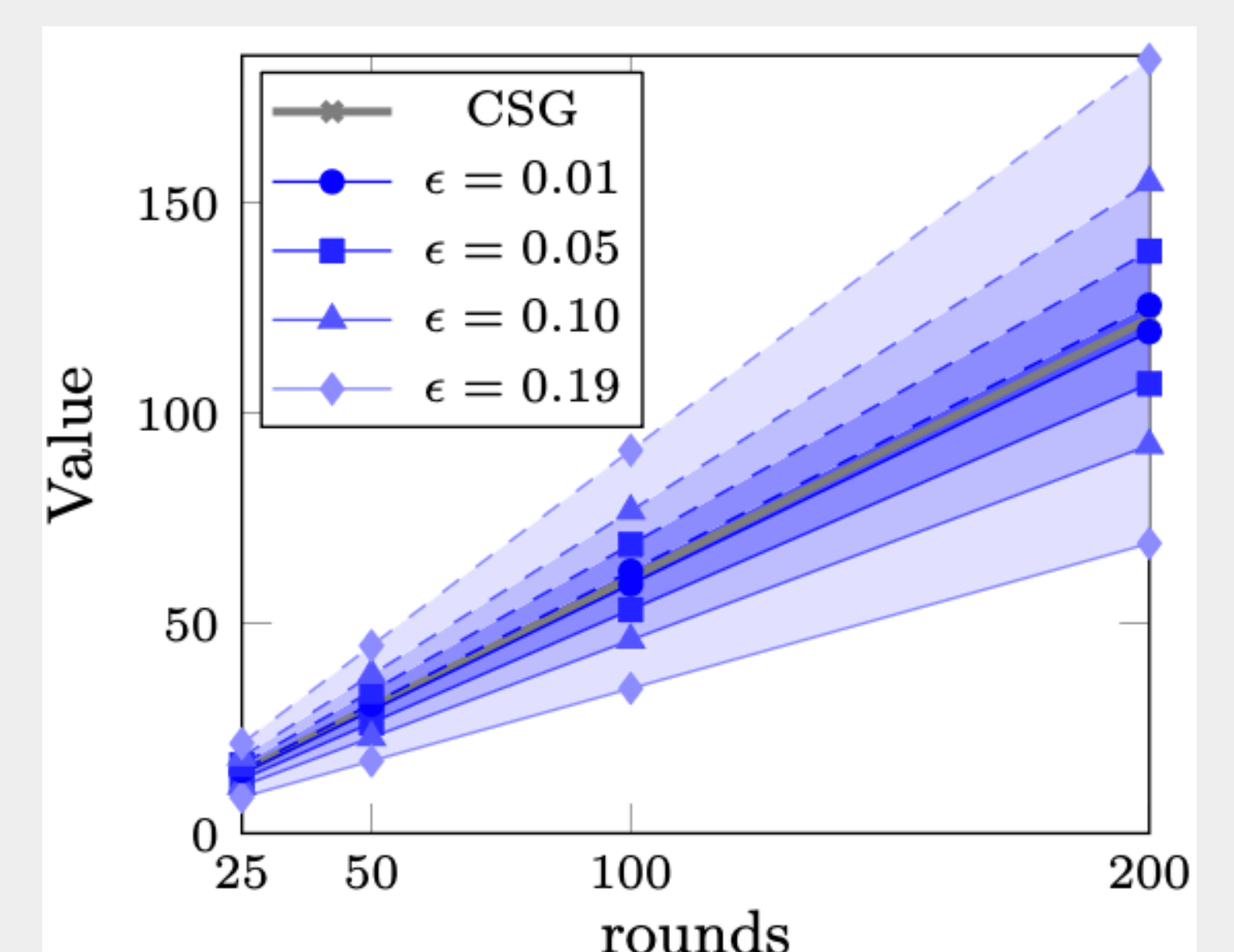
- Adversarial expansion (ICSG $\mathcal{G} \rightarrow$ CSG $\mathcal{G}^A$) is **value-preserving**
 - Player-first = nature-first** semantics: Equivalent robust value regardless of whether players or nature moves first
 - allows **robust dynamic programming**
 - i.e., fix nature's choice of P first, then solve induced CSG
- Enables algorithm to compute robust value of the ICSG by reusing CSG algorithms (without building $\mathcal{G}^A$ explicitly!)

RVI/RBI update for state $s \in S$ in $\mathcal{G} = (N, S, \bar{s}, A, \Delta, \check{P}, \hat{P})$

- for all** $a \in A(s)$ **do**
- $P_{sa}^* \leftarrow \text{SOLVEINNERPROBLEM}(s, a, V_{prev}, \check{P}, \hat{P})$
- end for**
- $\mathcal{Z} \leftarrow \text{CONSTRUCTNFG}(P_{sa}^*, V_{prev})$
- $V_{next}[s] \leftarrow \text{SOLVENFG}(\mathcal{Z})$

Experimental Results

- Implementation in **PRISM-games**
- CSG benchmarks augmented with transition uncertainty
- Zero-sum ICSGs:** similar scaling to non-robust CSGs
- millions of states & transitions possible
 - < 2× overhead over CSGs
- Nonzero-sum ICSGs:** more expensive but tractable for many instances



ICSG values vs. game size in *Intrusion Detection*

Future work

- Non-rectangular / non-polytopic** uncertainty
- Richer **temporal** objectives (e.g., LTL)
- Learning** robust learned controllers